CS 422 Assignment 3: TCP Congestion Control (Option 1 — Kernel Module)

Name: Kush Khanna **Repository:** https://github.com/krash3125/cs422-assignment-2 (branch `assignment-3`)

All numbers below come from one run of `./run_a3.sh`, stored in `results_a3/20261024-151559/`. Public paths change, so a re-run reproduces the method, not the exact numbers.

1. How QAIMD updates the congestion window

`tcp_qaimd.c` implements the QAIMD algorithm from Assignment 2 Q3(b): it anchors `cwnd` to the estimated BDP (btl_bw × min_rtt) and keeps the standing queue small. It registers as CC `"qaimd"` (`tcp_qaimd_ops`) and is selected by the Python client with `setsockopt(TCP_CONGESTION, "qaimd")` before `connect()`; `cc_used = qaimd` in all 94 successful tests. It uses the `.cong_control` hook (like BBR), so it owns every `cwnd` update.

Per ACK (`qaimd_main`): it tracks `min_rtt` (10 s windowed min), the per-round min RTT, a windowed-max delivery rate `btl_bw`, and per-round retransmit/delivered counts. Queue q = round_min_rtt − min_rtt; a loss counts as **congestive** only if $q > q_{\text{thr}}$ or the round loss ratio > 2%.

Per round (`qaimd_update_Round`): - **Startup:** double `cwnd` until congestive loss, a sustained queue, or a bandwidth plateau; then drop to the BDP. - **Avoid:** congestive loss → `cwnd ← 0.7·cwnd`; queue over threshold → proportional back-off; below BDP → catch up by `(bdp−cwnd)/4`; else `+1 MSS`. - **RTO:** `cwnd = 4`, restart in startup capped at the last BDP.

2. Initial state, parameters, and changes from the pseudocode

Parameter	Value	Purpose
initial state / cwnd	`STARTUP` / 10 MSS	exponential bandwidth search
`QAIMD_BETA`	0.7	multiplicative decrease on congestive loss
`QAIMD_LOSS_TOL`	2.0%	loss ratio counted as congestion
`QAIMD_MIN_CWND`	4 MSS	window floor
`q_thr`	clamp(0.15 · min_rtt, 3–15 ms)	target standing queue
`QAIMD_PACE_GAIN`	1.1×	pacing multiplier

Changes from the pseudocode: (1) floating-point constants done in fixed-point integer math (kernel forbids FP); (2) loss ratio measured in segments (`total_retrans/delivered`) not bytes; (3) `btl_bw` is a time-windowed max rather than a 10-slot ring buffer (fits the 104-byte state limit); (4) `min_rtt` refreshed via the ProbeRTT drain, not an overwrite; (5) delay back-off capped at 30%; (6) uses the 2-argument `cong_control(sk, rs)` hook of the 6.8 kernel.

3. Kernel environment, build, and reproduction

- **Kernel:** `6.8.0-147-generic` (aarch64), Ubuntu 24.04.5 LTS, in a Multipass VM on macOS (a real kernel that can load a module; Docker Desktop cannot). Default qdisc `fq_codel`; recorded in `metadata.json`.
- **Requirements:** `CONFIG_TCP_CONG_ADVANCED=y` and `tcp_bbr`.
- **Build & load:**

```
sudo apt-get install -y build-essential linux-headers-$(uname -r) \
     python3-numpy python3-pandas python3-matplotlib
make            # builds tcp_qaimd.ko (KDIR=/lib/modules/$(uname -r)/build)
make load       # insmod + show available CC: reno cubic bbr qaimd
```

- **Run everything:** `sudo ./run_a3.sh` → writes `results_a3/<RUN_ID>/{samples.csv, tests.jsonl, tables, plots}`.

4. Method

8 of 10 requested public iperf3 servers were selected (`destinations.json`); each tested with **qaimd, cubic, reno, bbr** for **10 s**, sampled every **200 ms**. Loss signal logged: retransmitted segments/bytes, RTO events, `ca_state`. **3 repetitions**, algorithm order rotated each rep (rep1 qaimd→cubic→reno→bbr; rep2 cubic→reno→bbr→qaimd; rep3 reno→bbr→qaimd→cubic); the four run back-to-back per destination. **94 of 96 tests succeeded** (two timed out after retries). Representative destination: **`84.17.41.11`** (Seattle, min RTT ~48 ms). Code: `run_experiments.py`, `analyze.py`.

5. Results

Aggregate throughput and RTT (all destinations/reps; `tables/a3_algo_summary.md`, p95 via `quantile(0.95)`):

CC	runs	goodput mean (Mbps)	goodput p95	srtt median (ms)	RTT inflation	retrans %
qaimd	23	103.5	186.8	**67.8**	**1.09**	**0.24**
cubic	24	139.6	254.6	114.3	1.61	0.31
reno	24	144.2	253.8	104.2	1.54	0.63
bbr	23	157.6	283.0	114.5	1.68	0.43

Plots (representative destination, all four algorithms — code: `analyze.py`):

Observed differences. CUBIC, Reno, and BBR push `cwnd` past the BDP, so the excess sits in the bottleneck queue: they reach higher goodput (140–158 Mbps mean) but inflate RTT 1.5–1.7× (sawtooth cwnd, rising srtt). QAIMD holds `cwnd` near the BDP, so srtt stays near min_rtt (inflation 1.09×) with the lowest retransmit rate (0.24%), at the cost of ~30% throughput — it gives the lowest srtt on 7 of 8 destinations (`tables/q3_winners.md`). On the long, thin paths (Sydney, Singapore) no algorithm builds a queue (inflation about 1.0), so QAIMD's latency advantage disappears and its conservatism costs throughput — its main limitation.

6. Use of AI tools

I used Claude Code to translate the QAIMD pseudocode into a kernel module, check it against the 6.8 `tcp_congestion_ops` interface, build/load it in the Multipass VM, and draft this report. I verified it by loading the module, confirming `qaimd` is selected on the socket, and checking server bytes match `tcpi_bytes_acked` to 0.04%. I can explain all of it in the oral exam.

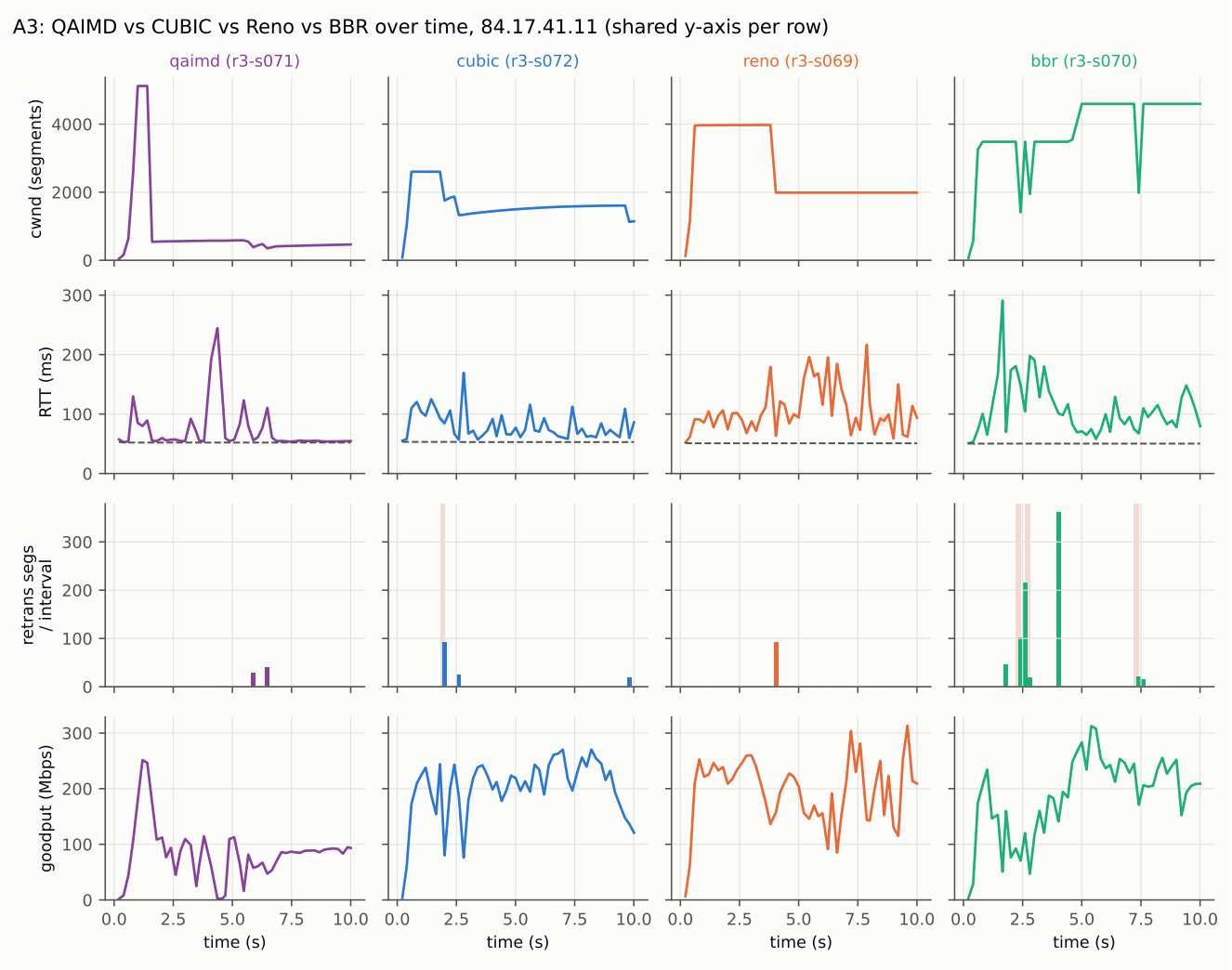


Figure 1: Time series: cwnd, RTT, loss, goodput.

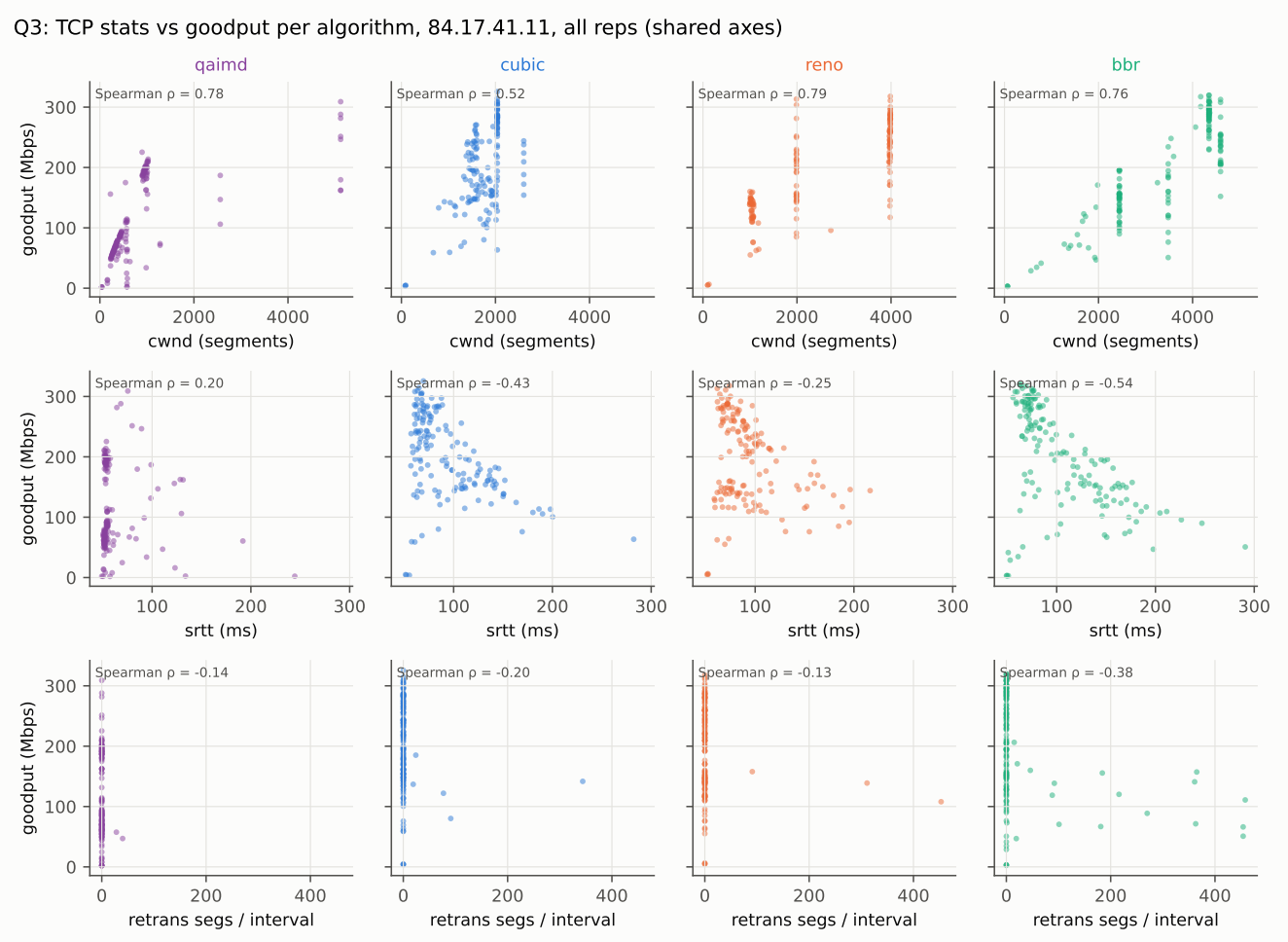


Figure 2: Scatter: cwnd, srtt, retransmits vs goodput.